

# Performance Evaluation of an AWS-Based Image Processing Application

Pasquale Luca Tommasino\* and Francesco Proietti†  
*Cloud Computing, MSc. in Data Science, Sapienza University of Rome*

(Dated: January 8, 2026)

This study investigates the behavior of an image processing application deployed on AWS, designed to convert color images into grayscale images using a serverless architecture. The system integrates several AWS services, including AWS Lambda, Amazon API Gateway, Amazon S3, and Amazon Cognito, while Amazon EC2 instances are used to generate the workload during the testing phase. The main objective of this study is the quantitative evaluation of system performance and scalability under different workload conditions. The workload is generated using Locust, simulating an increasing number of concurrent users, while performance metrics are collected through Amazon CloudWatch. The analysis focuses on the behavior of the core AWS Lambda function within the overall architecture, evaluating metrics such as execution time, number of invocations, and concurrency levels. The experimental results highlight how the system responds to variable workloads, providing insights into its scalability properties and performance limitations.

Keywords: aws, performance evaluation, locust, metrics, cloud computing

## I. INTRODUCTION

Serverless computing has emerged as a widely adopted paradigm for deploying scalable cloud applications, allowing developers to focus on application logic while delegating infrastructure management and scaling to the cloud provider. Among serverless services, AWS Lambda is extensively used for event-driven and compute-intensive workloads, including image processing applications. However, understanding the performance and scalability characteristics of such applications under varying workload conditions remains a crucial aspect of system design. This study investigates the performance behavior of a serverless image processing application deployed on AWS, designed to convert color images into grayscale images. In line with the course objectives, the focus is not on the complexity or accuracy of the image processing algorithm, but on the quantitative evaluation of system performance and scalability. The application is built using multiple AWS managed services, including AWS Lambda, Amazon API Gateway, Amazon S3, and Amazon Cognito, forming a realistic cloud-based architecture. Since the performance evaluation is primarily focused on the Lambda function, a simplified version of the architecture is adopted for the experimental analysis, while the complete system architecture is presented later. To assess the scalability of the system, a structured performance evaluation is conducted under controlled workload conditions. The analysis concentrates on the core computational component

of the application, namely the Lambda function responsible for image conversion, which represents the main source of computational load. Workloads are generated using Locust, simulating different levels of concurrent users, while performance metrics are collected through Amazon CloudWatch. By isolating the behavior of the image conversion function and analyzing key metrics such as execution duration, invocation count, concurrency, and error rates, this study provides insights into how serverless applications respond to dynamic workloads. The results highlight the strengths and limitations of the serverless execution model within the constraints of an academic cloud environment, contributing to a deeper understanding of performance evaluation in cloud-based systems.

## II. MATERIALS AND METHODS

### A. Image Processing and Input Data

The image processing task considered in this study consists of converting color images into grayscale images, with the primary objective of evaluating the performance and scalability of the underlying serverless architecture rather than the complexity of the image processing algorithm itself. The input images were collected from multiple publicly available datasets hosted on Kaggle. These datasets include images with heterogeneous characteristics in terms of content, resolution, and file size, allowing the generation of realistic and diverse workloads. To support a structured performance evaluation, the images were preprocessed and categorized according to their file size using a dedicated Python script. The script automatically organizes

---

\* Email address: tommasino.1912107@studenti.uniroma1.it;  
Student ID: 1912107

† Email address: proietti.1873188@studenti.uniroma1.it;  
Student ID: 1873188

the images into separate directories based on predefined size ranges, which are used to define different workload classes. Specifically images with a size:

- Between 100 KB and 220 KB were classified as **small**,
- Between 1 MB and 3 MB were classified as **medium**,
- Approximately 5 MB were initially classified as **large**.

To simplify workload generation and request handling, image filenames were renamed according to their size category, embedding the category label directly into the filename. This allowed the testing framework to select input images programmatically without relying on directory-level metadata. However, due to resource constraints imposed by the AWS Academy Learner Lab environment and to avoid excessive load that could lead to account suspension, the large image category was excluded from the experimental evaluation. Preliminary tests showed that processing large images under concurrent workloads caused excessive resource consumption and account throttling. Consequently, the performance analysis presented in this study focuses on small and medium images, which are sufficient to evaluate the scalability and performance trends of the deployed system while remaining compliant with the platform limitations.

### B. AWS Architecture Application

The application is built on a serverless architecture deployed entirely on AWS, designed to support an image processing workflow for converting color images into grayscale images. The architecture, shown in Figure 1, integrates multiple AWS services and represents the complete operational environment of the application under normal usage conditions. The interaction starts from the client side, where users access a static web interface hosted on an Amazon S3 bucket. User authentication is managed through Amazon Cognito, which provides secure access control to the application. Once authenticated, users interact with the backend through Amazon API Gateway, which acts as the main entry point for all application requests. API Gateway forwards incoming requests to a dedicated AWS Lambda function **Lambda4API**, responsible for handling application logic related to request validation, metadata management, and interaction with storage services. Input images are stored in a source Amazon S3 bucket, while the processed grayscale images are saved in a

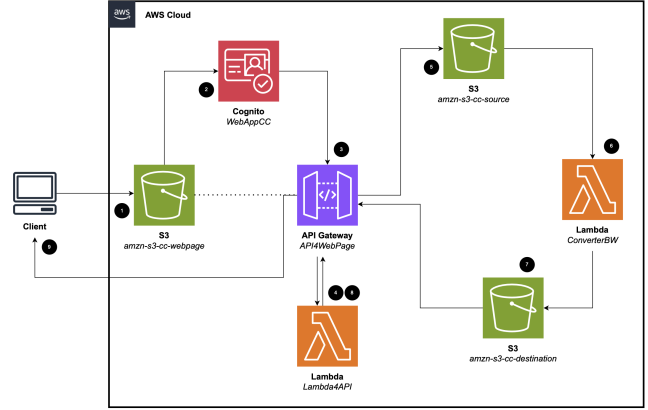


Figure 1: Overall AWS application architecture for the image processing workflow.

separate destination bucket. The actual image conversion is performed by a second AWS Lambda function **ConverterBW**, which represents the core computational component of the system. This architecture reflects a realistic end-to-end deployment, where multiple managed services cooperate to provide authentication, request handling, storage, and computation. As such, it may naturally introduce additional latency and potential bottlenecks due to service orchestration, network overhead, and service-level limits. However, while this full architecture represents the real application workflow, the performance evaluation conducted in this study does not aim to characterize the end-to-end behavior of the entire system. Instead, the analysis focuses on the core image processing component, namely the **ConverterBW** Lambda function. For this reason, performance experiments were carried out using a separate, simplified testing architecture specifically designed to stress the image conversion function in isolation. This approach allows the evaluation to focus on computational scalability and execution performance without interference from external components that are not central to the study objectives. The details of the performance evaluation setup and the rationale behind this methodological choice are discussed in the following subsection. For further details and to replicate the experiments described in this subsection, please refer to the GitHub repository.

### C. Performance Evaluation Methodology

The performance evaluation conducted in this study is specifically designed to assess the scalability and execution behavior of the core image processing component of the system, namely the AWS Lambda func-

tion responsible for converting images into grayscale **ConverterBW**. Rather than evaluating the end-to-end application workflow described in the previous section, the analysis focuses on the isolated behavior of this function under controlled and reproducible workload conditions. To achieve this goal, a dedicated testing architecture was deployed (Figure 2), separate from the production-like application architecture. Due to the different architecture the original Lambda function was rearranged in order to manage the simulated requests properly. This testing setup directly targets the rearranged **TestConverterBW** function through a simplified invocation path, avoiding the accumulation of traffic, latency, and potential bottlenecks introduced by ancillary services such as user authentication, web interfaces, or multi-step request orchestration. This methodological choice ensures that the observed performance metrics primarily reflect the computational characteristics of the Lambda function itself. Workload generation was performed using Locust on an EC2 instance. In this way even requests are confined within the perimeter of AWS. This is necessary due the limitations of the AWS Academy Learner Lab. Locust works based on a script, which was configured to simulate increasing numbers of concurrent users issuing image processing requests. Each user sends a request containing the file name of the image to be processed to the API Gateway. The image name is randomly chosen within the small images and the medium images with the following probabilities:

$$\mathbb{P}(X = x) = \begin{cases} 0.6, & x = \text{small}, \\ 0.4, & x = \text{medium}. \end{cases}$$

The API gateway transmits the payload and invokes the **TestConverterBW** function, which performs the grayscale conversion of the chosen image stored in a source S3 bucket. After the processing stage, the lambda function saves the image in a destination S3 bucket and returns a message containing a string that communicates the success or failure of the operation.

Different workload profiles were defined to evaluate system behavior under varying levels of concurrency and request intensity. In particular, the workload generated with Locust follows a staged profile designed to reproduce realistic and controlled load variations over time shown in Figure 3. The test execution is divided into multiple phases, allowing the observation of both transient and steady-state system behavior. Specifically, the workload consists of an initial worm-up phase lasting 60 seconds with 5 concurrent users, aimed at reducing cold-start effects. This phase is followed by a ramp-up period from 60 to 240 seconds, during which the number of concurrent users gradually increases to 30. A steady period is then maintained from 240 to 480 seconds with 30 concurrent users, enabling the analysis of sustained load conditions. Subsequently, the workload is progres-

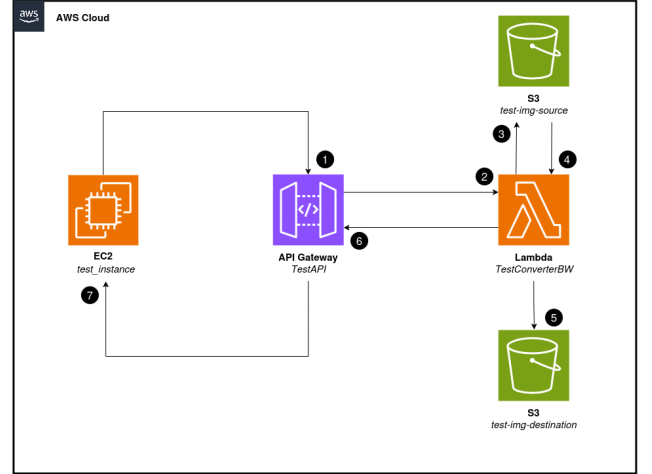


Figure 2: AWS Architecture of performance evaluation

sively reduced through a ramp-down phase from 480 to 600 seconds, decreasing the number of users back to 5, followed by a final ramp-down from 600 to 660 seconds, where the load is reduced to zero.

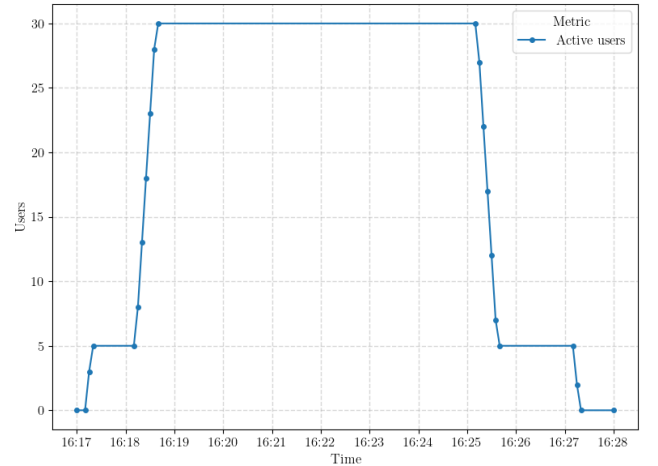


Figure 3: Number of Users active over time

Both **TestConverterBW** Lambda function and Locust-file script are shown in Appendix B. This staged workload design allows the evaluation of Lambda performance under increasing load, steady high concurrency, and decreasing demand, providing insights into scalability dynamics and resource adaptation behavior. Performance metrics were collected each minute using Amazon CloudWatch, which provides fine-grained monitoring of AWS Lambda executions. The analysis focuses on key system-level metrics such as execution duration, number of invocations, concurrency levels, and error rates. In addition, user-oriented performance indica-

tors were collected from the workload generator using Locust, including request rates and client-side response time statistics. The combined analysis of CloudWatch and Locust metrics enables a comprehensive evaluation of both backend execution behavior and user-perceived performance. This methodology allows for a clear separation between application level complexity and function level performance, while also capturing end-to-end effects, providing deeper insights into the scalability properties and limitations of the serverless execution model adopted in this study. Despite all the measures taken to avoid exceeding AWS Academy Learner lab limits, at the end of the first experiment we were inexplicably banned. In the next section the results of this single test are presented. Although we could not run other experiments for a rigorous analysis by averaging the results, the metrics collected in this single test still give us a good overview of the behavior of the Lambda function.

### III. RESULTS AND DISCUSSION

This section reports and discusses the results of the performance evaluation of the AWS Lambda function under the staged workload described in Section Section IIC. The analysis follows a multi-perspective approach, combining (i) native AWS Lambda metrics collected via CloudWatch, (ii) System-oriented indicators derived from CloudWatch counters, and (iii) User-oriented performance metrics measured at the client-side<sup>1</sup>

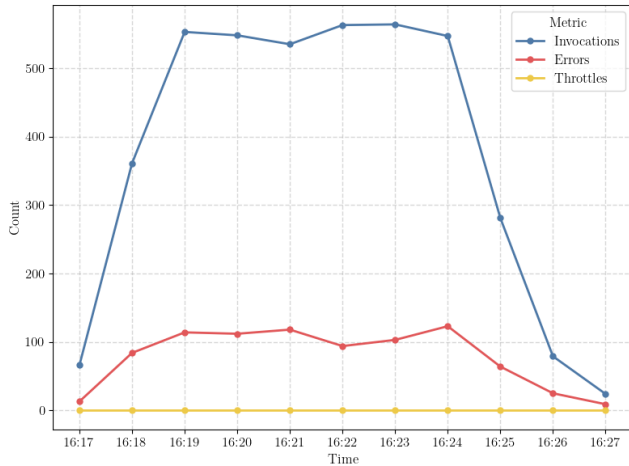


Figure 4: CloudWatch Invocation Metrics: Invocations, Errors, and Throttles over time.

Figure 4 presents the main invocation-related metrics collected from CloudWatch, namely **Invocations**, **Errors**, and **Throttles**. The invocation rate closely follows the imposed workload profile, increasing during the ramp-up phase and remaining high during the plateau, which confirms that the load generator effectively stresses the system. Errors are observed throughout the experiment but remain consistently lower than the total number of invocations, indicating that the function preserves operational continuity under sustained load. These errors are caused by AWS Academy Learner Lab restrictions. In fact the error rate could be easily decreased by expanding the memory of the Lambda function. We have also included two tables in the Appendix A that provide a detailed overview of the user-oriented metrics collected by Locust. Notably, **Throttles** are equal to zero for the entire duration of the experiment, showing that the AWS Lambda concurrency limits are never reached and that no requests are rejected by the platform. The execution performance of the function is analyzed through the CloudWatch metric **Average Duration**, shown in Figure 5.

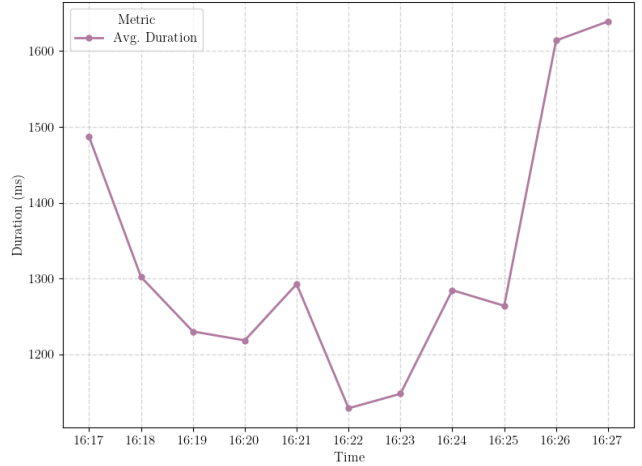


Figure 5: CloudWatch Performance Metric: **Average Duration** (ms) over time, highlighting warm-up effects and steady-state stability.

Higher execution times are observed during the initial phase of the experiment, which can be attributed to cold-start effects and the limited reuse of execution environments. Once the system reaches steady-state conditions, the average duration stabilizes within a narrow interval (approximately 1.2–1.3 seconds), indicating stable and predictable execution behavior under sustained concurrency. During the ramp-down phase, the average duration increases again, which is consistent with a reduction in invocation frequency and a renewed impact of cold starts (AWS (2026)).

<sup>1</sup> User-oriented performance metrics are evaluated on the performance evaluation architecture shown in Figure 2.

Scalability properties are examined by relating the imposed workload to the concurrency metric `ConcurrentExecutions`. Figure 6 jointly reports the number of active users generated by Locust and the corresponding number of concurrent Lambda executions observed via CloudWatch. During the ramp-up and plateau phases, concurrency increases and then stabilizes, while it decreases during ramp-down, closely mirroring workload variations. This behavior confirms that AWS Lambda dynamically adapts the number of execution environments to the incoming request rate, providing elastic scalability without manual intervention.



Figure 6: Scalability analysis: Locust active users and CloudWatch `ConcurrentExecutions` for showing dynamic adaptation to workload variations.

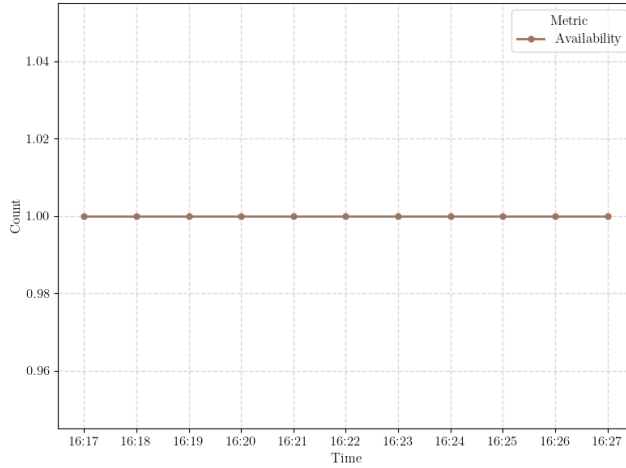


Figure 7: System-oriented `Availability` computed from CloudWatch. Availability remains equal to 1 due to the absence of throttling.

Following a system-oriented evaluation perspective, availability is derived from CloudWatch counters according to:

$$A(t) = 1 - \frac{\text{Throttles}(t)}{\text{TotalRequests}(t)}$$

In this analysis, `TotalRequests(t)` is approximated by `Invocations(t)`, representing the number of requests that effectively reach the Lambda function within each aggregation window. Figure 7 shows that availability remains equal to 1 for the entire duration of the experiment. This result directly follows from the absence of throttling events and indicates that, under the tested workload, the platform consistently accepts all incoming requests.

Throughput is computed as the rate of successfully served requests:

$$T(t) = \frac{\text{Invocations}(t) - \text{Errors}(t) - \text{Throttles}(t)}{\Delta t}$$

where  $\Delta t$  denotes the CloudWatch aggregation interval. The resulting throughput, reported in Figure 8, increases during the ramp-up phase, remains approximately stable during the plateau, and decreases during ramp-down. Given the absence of throttling and the relatively small number of errors, throughput closely follows the invocation rate, suggesting that system capacity is not saturated in the explored workload regime.

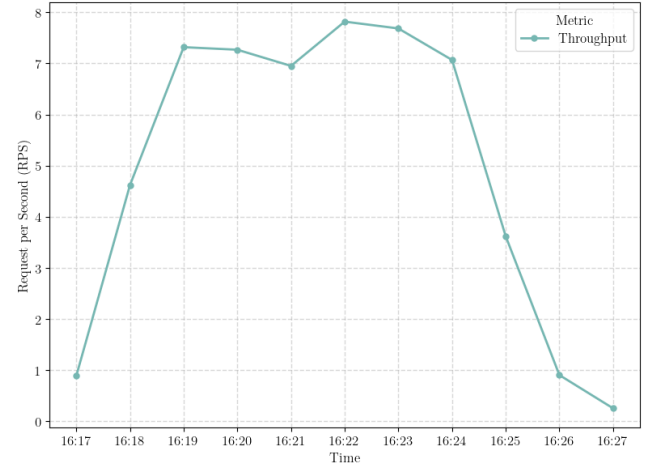


Figure 8: System-oriented Throughput (requests/second) computed from CloudWatch.

User-oriented performance is finally assessed through client-side measurements collected by Locust. Figure 9 reports the response time percentiles (p50 and p95), which capture the end-to-end latency experienced by users, including network delays, API Gateway processing, and Lambda execution. These metrics complement the CloudWatch `Duration`, which only accounts for execution time within the Lambda runtime.

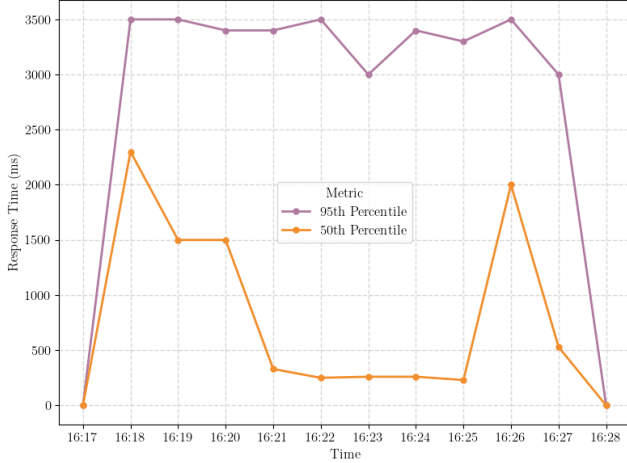


Figure 9: User-oriented response time measured by Locust: p50 and p95 percentiles over time, representing end-to-end latency.

The combined analysis of system-oriented and user-oriented metrics enables a more comprehensive assessment of both backend efficiency and quality of service. In addition to response time, the load generation behavior and request reliability are analyzed through the Requests Per Second (RPS) and failures per second metrics provided by the Locust report, shown in Figure 10.

The RPS curve closely follows the staged workload profile, increasing during the ramp-up phase, remaining stable during the plateau, and decreasing during ramp-down, confirming that the intended load was correctly applied. Failures per second slightly increase during periods of sustained load but remain limited with respect to the achieved RPS, indicating that the system maintains stable behavior under the tested conditions. These failures are consistent with the errors observed in CloudWatch metrics and are primarily attributable to the constraints of the AWS Academy Learner Lab, rather than to platform saturation or scalability limitations.

Overall, the experimental results indicate that `TestConverterBW` is able to sustain the imposed workload with stable execution times during steady-state operation, elastic concurrency adaptation, and no ob-

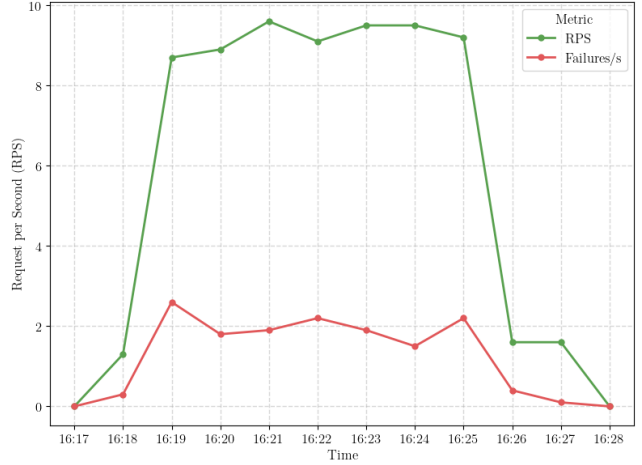


Figure 10: Client-side load metrics from Locust: Requests Per Second (RPS) and failures per second over time.

servable throttling phenomena. Errors remain limited with respect to the total number of invocations, while system-oriented indicators confirm full availability and throughput trends consistent with the staged workload profile. These findings collectively suggest that, within the tested conditions, the AWS Lambda platform provides effective scalability and reliable performance.

#### IV. CONCLUSIONS

This paper presented a performance evaluation of a serverless image processing workload deployed on AWS, focusing on the execution behavior and scalability of a single AWS Lambda function dedicated to grayscale image conversion. Rather than assessing the end-to-end performance of the complete application architecture, the analysis deliberately isolated the core computational component in order to obtain reproducible and interpretable measurements, minimizing the influence of auxiliary services and architectural bottlenecks. The experimental results, obtained through a controlled workload generated with Locust and monitored using system-level metrics collected via CloudWatch, show that the `TestConverterBW` Lambda function maintains stable execution times under sustained load and dynamically adapts its concurrency level in response to workload variations. Performance fluctuations observed during warm-up and ramp-down phases highlight the impact of cold-start effects and execution environment reuse, while steady-state behavior remains consistent across concurrent executions. The absence of throttling events and the limited number of execution er-

rors indicate that, within the explored workload regime, the function operates well within the platform limits imposed by the experimental environment. It is important to note that the adopted evaluation methodology, including the isolation of the Lambda function and the use of controlled workload generation, was strongly influenced by the constraints of the AWS Academy Learner Lab environment. This environment provides limited computational resources and credits compared to a standard AWS account, and does not allow the reproduction of unrestricted production-level workloads. As a result, the findings of this study are not intended to support general claims about the suitability of AWS Lambda for image processing workloads in real-world production scenarios. Instead, they provide a coherent and reproducible performance characterization within the boundaries of an academic and resource-constrained context. Future work may extend this analysis by exploring higher concurrency levels, larger input images, or by performing an end-to-end performance evaluation of the complete application architecture on a less constrained cloud environment, in order to quantify the additional overhead introduced by orchestration mechanisms and auxiliary cloud services.

## REFERENCES

AWS, “Understanding lambda function scaling,” (2026).

E. Casalicchio, “Performance evaluation of cloud systems and services,” (2024), lecture slides, Cloud Computing Course, Sapienza Università di Roma.

## Appendix A: Locust tables

Table I: Request statistics obtained from Locust performance evaluation.

| Method | Name            | # Requests | # Fails | Average (ms) | Min (ms) | Max (ms) | RPS  | Failures/s |
|--------|-----------------|------------|---------|--------------|----------|----------|------|------------|
| POST   | convert__medium | 1600       | 851     | 2596.51      | 1084     | 4012     | 2.66 | 1.41       |
| POST   | convert__small  | 2504       | 0       | 454.86       | 111      | 2750     | 4.16 | 0.00       |
|        | Aggregated      | 4104       | 851     | 1289.81      | 111      | 4012     | 6.82 | 1.41       |

Table II: Response time percentiles obtained from Locust performance evaluation.

| Method | Name            | 50%ile (ms) | 70%ile (ms) | 80%ile (ms) | 90%ile (ms) | 95%ile (ms) | 100%ile (ms) |
|--------|-----------------|-------------|-------------|-------------|-------------|-------------|--------------|
| POST   | convert__medium | 2600        | 3000        | 3000        | 3500        | 3500        | 4000         |
| POST   | convert__small  | 190         | 230         | 1400        | 1400        | 1500        | 2700         |
|        | Aggregated      | 1300        | 2100        | 2500        | 3000        | 3500        | 4000         |

## Appendix B: Code

### 1. TestConverterBW

```

1  import json
2  import boto3
3  from io import BytesIO
4  from PIL import Image
5  import uuid
6  import os
7
8  s3 = boto3.client('s3')
9
10 SOURCE_BUCKET = "test-img-source"
11 DEST_BUCKET = "test-img-destination"
12
13 def lambda_handler(event, context):
14     try:
15         # File name from the API Gateway event
16         body = json.loads(event["body"])
17         filename = body["filename"]
18
19         # Image download from the source bucket
20         img_obj = s3.get_object(Bucket=SOURCE_BUCKET, Key=filename)
21         img_data = img_obj["Body"].read()
22
23         # Open the image and perform the grayscale conversion
24         img = Image.open(BytesIO(img_data))
25         bw_img = img.convert("L")
26
27         # Save the processed image
28         buffer = BytesIO()
29         bw_img.save(buffer, format=img.format)
30         buffer.seek(0)
31
32         # Generate a unique name for the processed image
33         # and upload the image on the destination bucket
34         name, ext = os.path.splitext(filename)
35         output_filename = f"bw_{name}_{uuid.uuid4().hex}-{ext}"
36         s3.put_object(

```

## II

```
37     Bucket=DEST_BUCKET,
38     Key=output_filename,
39     Body=buffer,
40     ContentType=img_obj["ContentType"]
41 )
42
43     # Return the message to the API Gateway
44     return {
45         "statusCode": 200,
46         "body": json.dumps({
47             "message": "Image converted successfully",
48             "output_file": output_filename
49         })
50     }
51
52     # Return the error, if it occurs
53     except Exception as e:
54         return {
55             "statusCode": 500,
56             "body": json.dumps({
57                 "error": str(e)
58             })
59     }
```

## 2. Locustfile script

```
1  from locust import HttpUser, task, between, LoadTestShape
2  import json
3  import random
4
5  class ImageConversionUser(HttpUser):
6      """
7      Locust user that simulates calls to the image conversion API.
8      Behavior:
9      - Randomly selects a category ("small" or "medium") with 60/40 weights.
10     - Chooses a random filename from the corresponding list.
11     - Sends a POST request to the /convert endpoint with the filename in the JSON body.
12     """
13
14     # Random wait time between requests (in seconds)
15     wait_time = between(1, 3)
16     # Set the API Gateway host
17     host = "*****"
18     # List of available "small" images for the simulation
19     small_images = [f"small_img_{i}.jpg" for i in range(1, 103)]
20     # List of available "medium" images for the simulation
21     medium_images = [f"medium_img_{i}.jpg" for i in range(1, 32)]
22
23     @task
24     def convert_image(self):
25         """
26         Main task:
27         1. Randomly selects the image category (small / medium) with 60/40 weights.
28         2. Selects a random filename from the corresponding list.
29         3. Builds the JSON payload with the filename.
30         4. Executes a POST request to /convert, measuring response time and success.
31         """
32         category = random.choices(["small", "medium"], weights=[0.60, 0.40], k=1)[0]
33         # Select filename based on the category
34         if category == "small":
35             filename = random.choice(self.small_images)
36         else:
37             filename = random.choice(self.medium_images)
```

```

37     # Request body in JSON format
38     payload = {
39         "filename": filename
40     }
41     # HTTP headers to indicate JSON body
42     headers = {
43         "Content-Type": "application/json"
44     }
45     # POST request to the /convert endpoint
46     # catch_response=True allows manually marking the request as success/failure
47     with self.client.post(
48         "/convert",
49         data=json.dumps(payload),
50         headers=headers,
51         name=f"convert_{category}",
52         catch_response=True
53     ) as res:
54         # If the response is not 200, mark it as a failure including the filename
55         if res.status_code != 200:
56             res.failure(
57                 f"Error during conversion of file '{filename}': "
58                 f"{res.status_code} {res.text}"
59             )
60         return
61
62 class StepLoadShape(LoadTestShape):
63     """
64     Custom load shape to generate a controlled traffic profile.
65     Stages:
66     - 0-60s: Worm-up (WU) with 5 users -1min
67     - 60-240s: Rump-up (RU) to 30 users -1/4min
68     - 240-480s: Steady (S) with 30 users -4/8min
69     - 480-600s: Ramp down (RD) to 5 users -8/10min
70     - 600-660s: Final ramp down to 0 users -11min
71     """
72     stages = [
73         {"duration": 60, "users": 5, "spawn_rate": 1},
74         {"duration": 240, "users": 30, "spawn_rate": 1},
75         {"duration": 480, "users": 30, "spawn_rate": 1},
76         {"duration": 600, "users": 5, "spawn_rate": 1},
77         {"duration": 660, "users": 0, "spawn_rate": 1},
78     ]
79     def tick(self):
80         """
81         Computes the current number of users and spawn rate
82         based on the total test runtime.
83         """
84         run_time = self.get_run_time()
85         for stage in self.stages:
86             if run_time < stage["duration"]:
87                 return stage["users"], stage["spawn_rate"]
88         return None

```